

Complete AI Engineer Training: Python, NLP, Transformers, LLMs, LangChain, Hugging Face, APIs

Section 1: Intro to AI Module: Getting Started

What does the course cover

AI Engineer Bootcamp designed to take learners from basic concepts to advanced, real-world AI applications through a structured, step-by-step approach. The program starts with foundational AI concepts and Python programming, then moves into natural language processing (NLP), large language models (LLMs), transformer architectures, and practical tools such as LangChain and vector databases like Pinecone.

The instructor, Ned, founder of 365 and an experienced Udeemy educator, introduces the course team—data science and computer science experts from leading European universities. Together, they have created a comprehensive and practical curriculum aimed at professional growth in the AI field.

The course covers:

- Fundamentals of artificial intelligence, its branches, tools, and career paths
- Python programming from basics to practical use cases like APIs and web scraping
- NLP as the foundation of generative AI
- Large language models and transformers
- Integrating LLMs into web applications using LangChain
- A final hands-on case study that consolidates all learned concepts

Learners are advised to follow the course sequentially, as each lecture builds on previous material. The program includes downloadable resources such as notes, PDFs, exercises, and notebooks to reinforce learning. Completing exercises is strongly encouraged to strengthen practical skills.

Students who complete 50% of the course and announce it in the Q&A section receive a bonus: exclusive access to a ChatGPT for Data Science course. The instructors actively encourage participation through questions and discussions in the Q&A section and express enthusiasm for learners to begin the journey.

Natural vs Artificial Intelligence

Natural intelligence is demonstrated through activities such as driving, solving complex mathematical problems, writing poetry, and mastering music. These abilities are not present at birth and are developed over time through learning. Learning is possible because the human brain can observe, process, and apply large amounts of information.

Intelligence is defined as the ability to acquire and apply knowledge and skills. Human intelligence has led to technological innovations that far surpass the tools available in earlier centuries.

Humans are effective tool builders and have continuously created tools to increase productivity. One of the most significant inventions was the Gutenberg printing press, developed in 1440, which transformed the way knowledge is shared. Despite its importance, the printing press cannot be considered an intelligent machine because it operates with fixed rules and cannot learn or adapt.

The concept of intelligent machines emerged much later. Inspired by human intelligence and brain processes, computer scientists sought to enable machines to acquire and apply knowledge. This effort led to the development of artificial intelligence.

Brief history of AI

Artificial Intelligence (AI) has evolved through several important scientific and technological breakthroughs over the past decades.

In **1950**, British mathematician and computer scientist **Alan Turing** posed the question “*Can machines think?*” in a scientific paper. His major contribution was shifting the discussion from philosophy to experimentation. He proposed the **Turing Test**, a practical method to evaluate machine intelligence. In this test, if a human interrogator cannot distinguish between a machine and a human through conversation, the machine is considered intelligent. This marked a foundational moment in AI research.

In **1956**, the term **Artificial Intelligence** was officially coined at the **Dartmouth Conference**, where AI became a formal academic field. Researchers explored whether machines could simulate aspects of human intelligence using concepts from neural networks, computation theory, and automata theory. From the beginning, AI research was inspired by how human intelligence works.

However, progress was slow due to limited computing power and lack of data. The **1960s and 1970s**, known as the **AI Winter**, saw reduced funding and interest in AI because expectations were not met.

A major breakthrough occurred in **1997** when **IBM’s Deep Blue** defeated world chess champion **Garry Kasparov**, demonstrating the potential of machine intelligence. In the late 1990s and early 2000s, rapid growth in internet usage and computing power created the necessary conditions for AI advancement. Massive amounts of digital data became available, and computational capabilities increased dramatically.

In **2006**, **Geoffrey Hinton** revived interest in neural networks through **deep learning**. Deep learning models, inspired by the human brain, require large datasets and high computational power—resources that were previously unavailable.

In **2011**, **IBM Watson** won the quiz show *Jeopardy!*, showcasing significant progress in **natural language processing**, enabling AI systems to understand and process human language.

In **2012**, researchers from **Stanford and Google**, including **Jeff Dean and Andrew Ng**, published influential work on large-scale unsupervised learning. Their deep neural network system successfully recognized complex visual patterns, such as identifying cats, marking progress in multilayer neural networks.

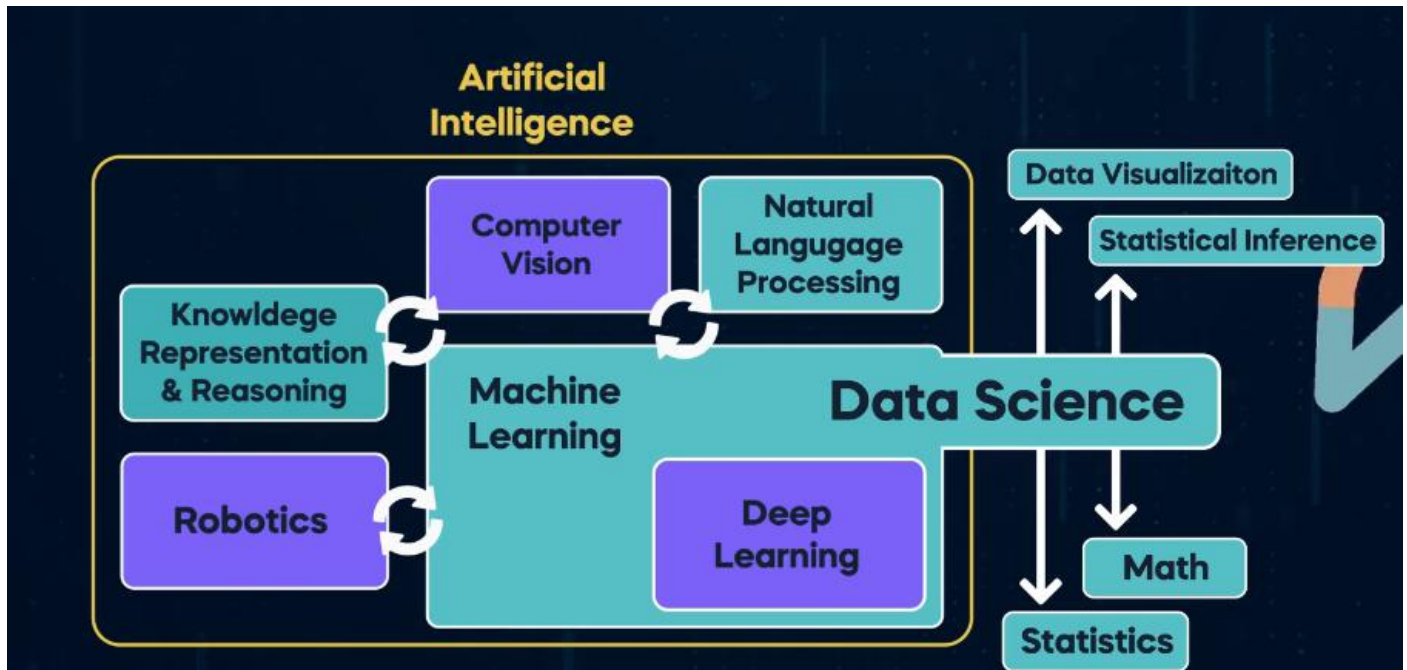
In **2017**, the **Google Brain** team introduced **Transformers**, a breakthrough model using self-attention mechanisms that greatly improved how machines process sequences like text.

In **2018**, **OpenAI** released the first version of **GPT (Generative Pre-trained Transformer)**, using Transformers to create large language models (LLMs). This innovation led to rapid improvements in generative AI and eventually to user-friendly systems like **ChatGPT**.

By **late 2022**, generative AI became widely accessible, marking a new era in artificial intelligence. Although the concepts may initially seem complex, they become clearer as learning progresses.

Demystifying AI, Data science, Machine Learning, and Deep Learning

The terms **Artificial Intelligence (AI)**, **Machine Learning (ML)**, and **Data Science** are closely related but have distinct meanings, and they should not be used interchangeably.



Artificial Intelligence is a broad discipline focused on making machines intelligent. Its goal is to enable systems to learn, adapt, and acquire new skills in a manner similar to human intelligence. AI encompasses several subfields, one of the most important being machine learning.

Machine Learning is a major subfield of AI that uses data to predict outcomes or make decisions. ML models learn complex patterns and dependencies in data that go beyond simple correlations. Similar to a mathematical function, input data is processed by a model to produce an output. For example, machine learning algorithms can analyze a user's movie-watching history to recommend new movies or evaluate financial transaction data to generate a credit score that predicts loan repayment behavior.

Data Science has a strong intersection with AI and machine learning but extends beyond them. Data scientists commonly work with machine learning algorithms, but they also use traditional statistical techniques such as data visualization, statistical inference, and exploratory data analysis. Valuable insights can often be extracted from data without relying on machine learning.

Data science integrates AI, machine learning, mathematics, statistics, and visualization techniques to derive business value from data. For instance, a data scientist may build a predictive model to forecast future customer orders or analyze and visualize customer behavior to uncover meaningful trends and insights.

Weak vs Strong AI

Artificial intelligence systems vary in capability and scope. Some AI systems are designed to perform a single, well-defined task. This form of intelligence is known as **Narrow AI**. Such systems can only do what they are specifically trained for and do not possess general reasoning abilities. Narrow AI is already widely used in daily life and business applications, such as recommendation systems that suggest movies based on viewing history.

More advanced systems fall under **Semi-Strong AI**. A notable example emerged in **2022** with the release of **ChatGPT and ChatGPT-3.5** by OpenAI. These systems demonstrated the ability to perform a wide range of tasks rather than a single specialized function. They can generate human-like responses, write jokes, proofread text, recommend actions, analyze and create images, generate programming formulas, and solve mathematical problems. Their conversational ability often makes them indistinguishable from humans, aligning closely with **Alan Turing's imitation game**, also known as the **Turing Test**. At this level, AI systems can be considered semi-strong due to their versatility and breadth of skills.

The next and most advanced stage is **Artificial General Intelligence (AGI)**, also referred to as **Strong AI**. AGI has not yet been achieved. According to **Sam Altman**, CEO of OpenAI, AGI would be the most powerful technology humanity has ever created. Such an AI would surpass human capabilities across many tasks and possess true general intelligence. Many researchers believe that a key requirement for AGI is the ability to create new scientific knowledge independently by synthesizing existing information to generate original discoveries and ideas.

Opinions differ on how close humanity is to achieving AGI, but some experts believe it could emerge in the near future. As progress continues toward this goal, it raises profound questions about the potential impact and ethical responsibilities associated with creating machines that can think, learn, and innovate more effectively than humans.

Section 2: Intro to AI Module: Data is essential for building AI

Structured vs unstructured data

Data is the core requirement for building any AI model.

There are two main categories of data: **structured** and **unstructured**.

Structured data is neatly organized into rows and columns (e.g., Excel sheets, databases), making it easy to store and analyze.

Unstructured data includes text, images, videos, and audio, which do not follow a fixed tabular format.

Approximately **80–90% of all global data is unstructured**.

In the past, structured data was seen as more valuable because it was simpler to process.

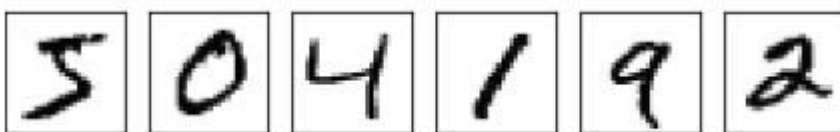
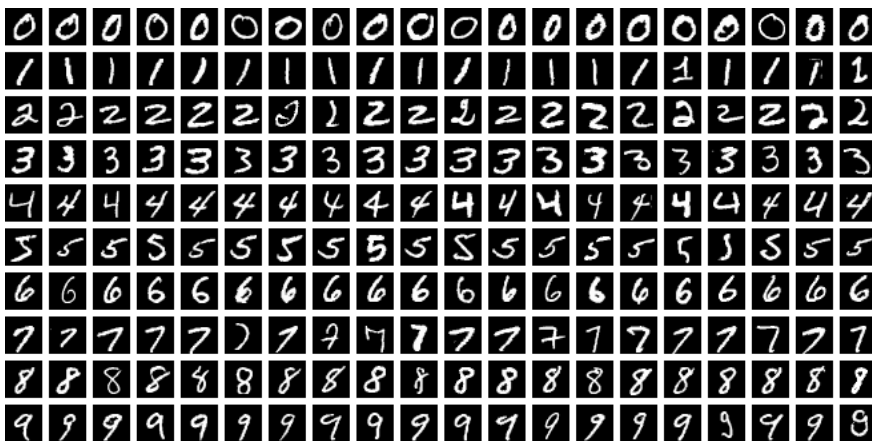
Modern AI techniques now enable valuable insights to be extracted from unstructured data.

Companies such as **Google and Meta** are leaders in leveraging unstructured data at scale.

Analyzing unstructured data allows businesses to gain deep insights from photos, videos, emails, and messages.

The next lesson focuses on **how AI systems interpret and learn from unstructured data**.

How we collect data



192	177	112	46	87	141	28	167	91	57
20	31	131	179	60	105	151	158	133	190
114	0	83	8	183	45	122	143	153	93
119	42	184	237	228	140	109	15	62	0
200	191	0	213	156	135	254	1	57	48
149	247	123	32	249	16	108	253	33	134
57	232	228	38	175	112	28	68	197	143
175	227	123	152	239	171	146	204	46	139
181	120	138	214	130	248	237	2	195	30
229	202	94	79	0	64	251	236	172	126

Summary: MNIST and How Computers Learn from Data

- The **MNIST dataset** is considered the “*hello world*” of machine learning and is widely used by beginners.
- It contains about **70,000 images** of handwritten digits (0–9), each sized **28×28 pixels** in grayscale.
- The goal is to train a machine learning model to **recognize digits**, even though each digit looks different due to variations in human handwriting.

How computers understand images

- Every image is made up of **pixels**, and each pixel has a grayscale value from **0 to 255**
 - 0 → white
 - 255 → black
- Computers store and process these pixel values in **binary form (0s and 1s)**.
- As a result, an image becomes a **numerical sequence**, not a visual object, from the computer’s perspective.
- Different handwritten versions of the same digit (like “3”) look different to humans but produce **similar numerical patterns** for the computer.

Learning with MNIST

- Machine learning algorithms learn by identifying **patterns and similarities** in these numerical representations.
- Through training, the model learns to distinguish between digits such as **3, 6, and 9**, based on their pixel-value patterns.
- MNIST demonstrates how **real-world information can be converted into data** that machines can process.

Beyond images

- Videos are sequences of images (frames), each made of pixels.
- Audio and speech can also be represented as numerical data.
- Ultimately, **all digital information is reduced to zeros and ones**, giving computers a structured way to learn.

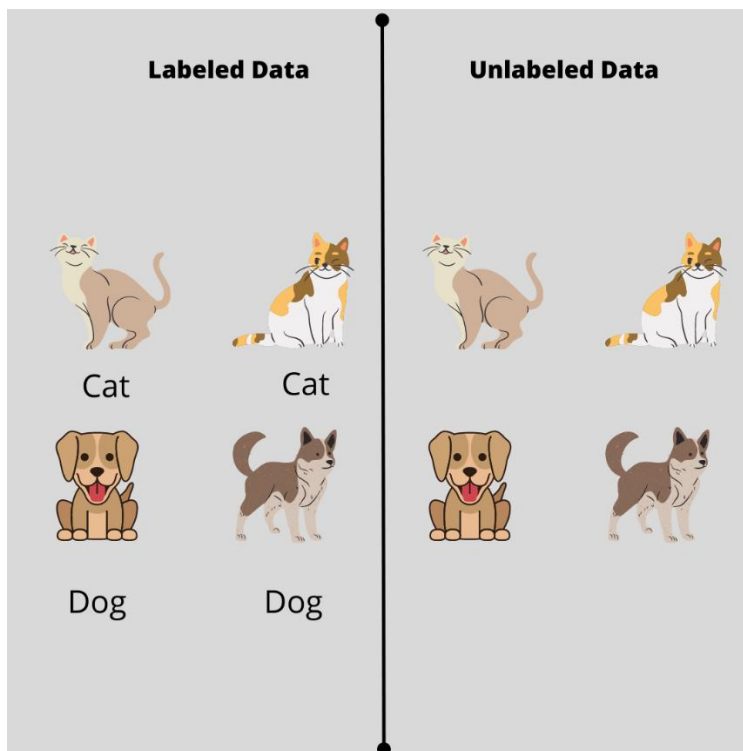
Human intelligence vs AI

- Humans naturally process massive amounts of sensory data simultaneously (sight, sound, touch, taste).
- AI researchers aim to replicate this capability using data from **sensors, images, videos, audio, text, social media, satellites, and web activity**.
- Data is collected through **APIs, web scraping, and big data analytics** to train and improve AI models.

Key principle

- “*Garbage in, garbage out*” — the **quality of data** directly determines the **quality of AI model outputs**.
- High-quality data is essential for building effective and reliable artificial intelligence systems.

Labelled and unlabelled data



Labeled vs Unlabeled Data in AI

- There are **two primary approaches to preparing data** for AI models: **labeled data** and **unlabeled data**.

Labeled data

- In labeled datasets, each data item is **manually tagged** with the correct category or outcome.
- Example: Reviewing **10,000 animal photos** and labeling each as *dog* or *not a dog*.
- The same approach applies to **text, audio, video**, and other data types (e.g., labeling YouTube comments as positive, negative, or neutral).
- Although **time-consuming and expensive**, labeled data produces **high-quality training data**.
- Models trained on labeled data are generally **more accurate, reliable, and effective in real-world applications**.

- The key trade-off is **higher cost and effort versus better performance**.

Unlabeled data

- Unlabeled datasets contain data **without predefined categories**.
- Modern machine learning techniques can learn patterns directly from **unstructured data** such as images, text, video, and audio.
- Example: Feeding the same 10,000 photos into a model **without labeling them beforehand**.
- The model is left to **discover patterns on its own**, reducing manual effort and cost.

Key takeaway

- You should clearly understand the **difference between labeled and unlabeled datasets**.
- Choosing between them involves balancing **data preparation effort, cost, and model performance**.

Metadata: Data that describes data

Data Growth and the Role of Metadata in AI

- The rapid **advancement of AI** is largely driven by the **digitalization of everyday life**.
- Growth in **online shopping, smartphones, cameras, social media, sensors, and IoT devices** has caused an **exponential increase in data generation**.
- Today's data is not only more abundant but also **higher in quality**, as seen in the improvement of modern smartphone photography compared to older devices.
- This explosion of **high-quality data** has strongly accelerated AI development.

Challenge of modern data

- Much of the data generated today is **unstructured** and **too large to be manually labeled**.
- Making sense of this massive volume of data requires additional context.

Metadata

- **Metadata is data about data**.
- It provides descriptive information such as **file type, author, creation date, usage details, and file size**.
- Example: In an image dataset (e.g., dog photos), metadata may include the photo name, capture date, photographer, and file size.

Importance of metadata

- Metadata is essential for **organizing, managing, and understanding data**.
- It is valuable across **structured and unstructured, labeled and unlabeled** datasets.
- In **large-scale datasets and complex AI systems**, metadata plays a critical role in enabling efficient data processing and analysis.

Section 3: Intro to AI Module: Key AI techniques

Machine Learning

The lesson transitions from general AI concepts to **machine learning (ML)**, emphasizing ML as the key driver of modern AI success.

ML is defined as designing systems that **learn and improve through trial and error using data**.

Core Analogy

- ML model = student
- Data scientist = teacher
- Training data = learning material
- With sufficient, high-quality data, the model learns patterns and performs well on new, unseen problems.
- Important insight: **a simpler model with abundant high-quality data can outperform a complex model with poor data.**

Key Concepts

- ML algorithms learn patterns from historical data.
- Performance improves with:
 - More relevant data
 - Better data quality
 - Updated algorithms
- Different ML models are suited for different problem types, so choosing the right model is the data scientist's responsibility.

Practical Example (Real Estate)

- Goal: build an app that predicts house prices.
- Requirement: large, well-structured historical dataset.
- Available features include:
 - house price
 - size
 - room/bedroom count
 - distance from city center
 - neighborhood, etc.
- The model learns a function **$y = f(x)$** :
 - **Input (x):** home characteristics entered by users
 - **Output (y):** predicted house price
- The trained model uses past transaction patterns to estimate prices for new properties.

- The example concludes with the app being successful.

Closing

- The lesson establishes what ML is, why data quality matters, and how ML models make predictions.
- Next topic preview: **types of machine learning models**.

Supervised, Unsupervised, and Reinforcement

Machine learning has three primary paradigms:

Supervised learning uses labeled data where correct outputs are known during training. The model learns to map inputs to outputs. It is mainly used for:

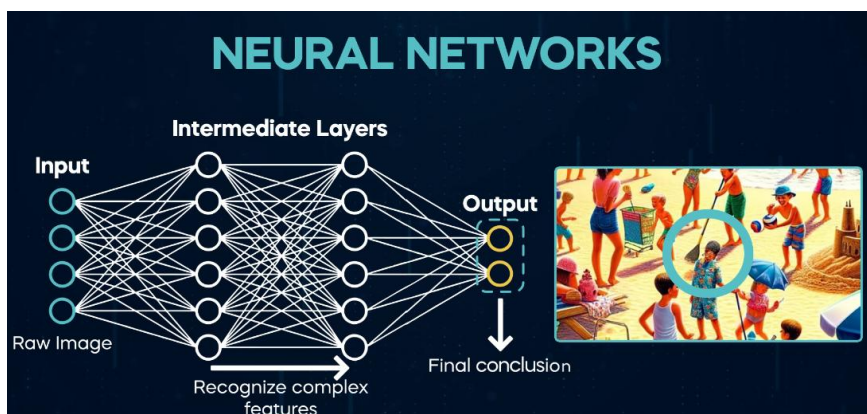
- **Classification** (e.g., detecting whether an image contains a dog)
 - **Regression/prediction** (e.g., estimating house prices from past sales data)
- The key idea is that the model learns from examples with explicit answers.

Unsupervised learning works on unlabeled data. The algorithm searches for hidden patterns and groups similar data points without being told what the groups represent. For example, given many animal images without labels, it can cluster similar animals together. This approach is valuable when labeling is expensive or when the structure of the data is unknown. Typical uses include customer segmentation and pattern discovery.

Reinforcement learning learns through trial and error to achieve a defined goal. Instead of labeled examples, the system follows rules, interacts with an environment, and improves based on feedback (rewards or penalties). A common application is recommendation systems, where the model adapts to user behavior over time by learning from interactions such as watching, skipping, or saving content.

Deep Learning

Deep learning is a subset of machine learning inspired by the way the human brain processes information in stages. It relies on **artificial neural networks (ANNs)** that progressively extract more detailed features from raw input data.

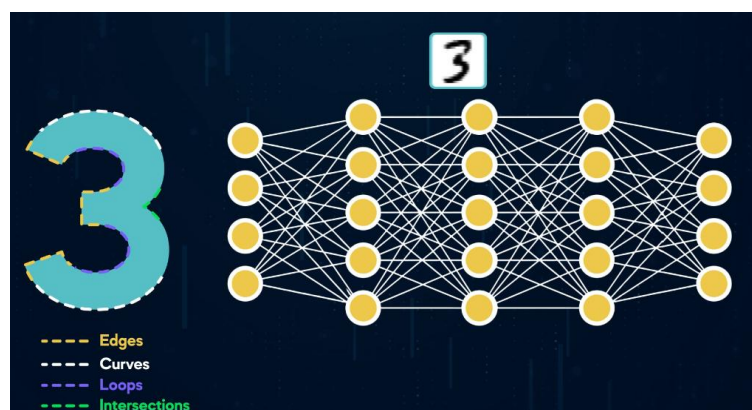
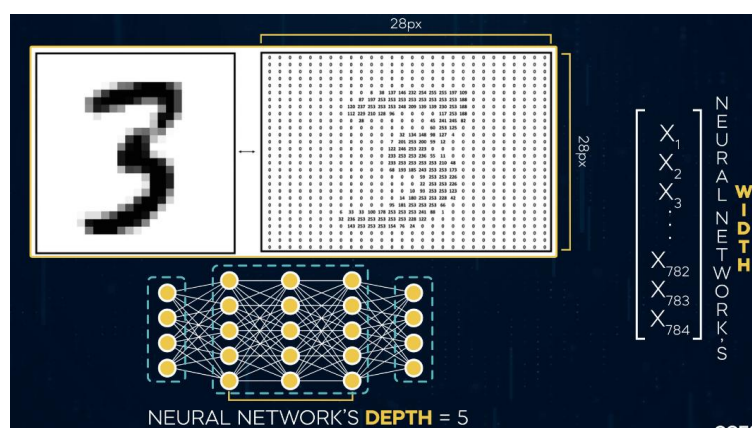
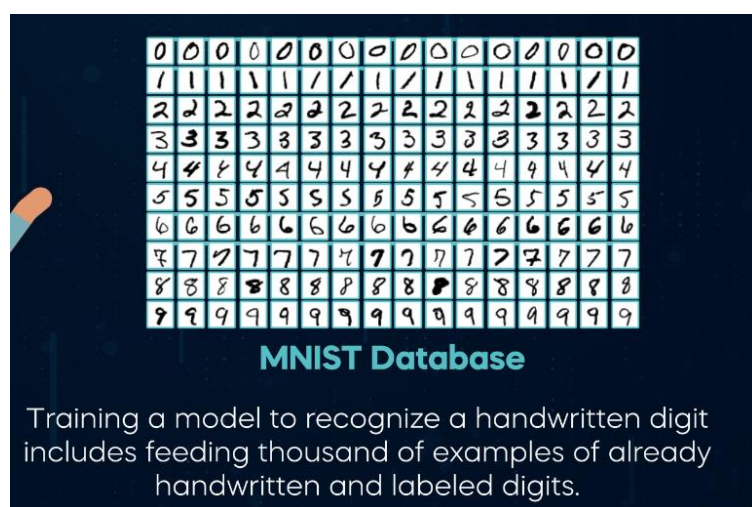


A neural network processes data layer by layer:

- **Input layer** receives raw data (e.g., pixels of an image).
- **Hidden layers** progressively detect more complex patterns.
- **Output layer** produces the final prediction.

This staged processing mirrors human perception: first we see the overall scene, then shapes, and finally fine details.

Each layer in a neural network consists of **neurons (nodes)** connected to the next layer. These connections apply mathematical transformations using **weights**, **biases**, and **non-linear functions**. Learning occurs by repeatedly adjusting these weights to minimize prediction errors.



Using the MNIST digit example:

- A 28×28 image becomes **784 input nodes**.
- Early hidden layers may detect **edges and curves**.
- Deeper layers combine these into **complex shapes**.
- The output layer determines which digit is present.

Key structural terms:

- **Width**: number of neurons in a layer
- **Depth**: number of layers in the network
- More layers increase learning capacity but require careful training.

Overall, deep learning works by transforming raw data through multiple levels of abstraction, enabling highly accurate pattern recognition in large, high-dimensional datasets.

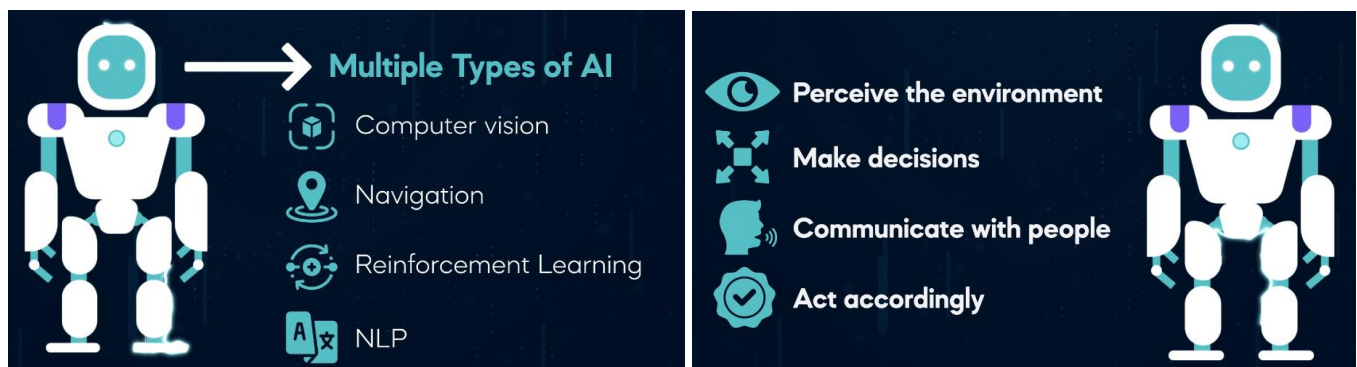
Section 4: Intro to AI Module: Important AI branches

Robotics

Robotics is the interdisciplinary field focused on designing, building, operating, and using robots capable of autonomous or human-like tasks.

Robotics integrates multiple engineering domains:

- **Mechanical engineering** → physical structure and movement
- **Electrical/electronics engineering** → power and control systems
- **Artificial intelligence** → decision-making and behavior
- **Sensors and cameras** → environmental perception



Modern robots typically combine several AI components rather than relying on a single model. A sophisticated robot may use:

- Computer vision for object detection
- SLAM for mapping and navigation
- Reinforcement learning for decision-making
- Natural language processing for human interaction

This integration allows robots to perceive, decide, communicate, and act effectively.



Current and emerging applications include factory automation, medical robotics, self-driving vehicles, agricultural robots, cleaning systems, space exploration robots, search-and-rescue machines, and

surveillance systems. Companies such as **Tesla**, under **Elon Musk**, are developing humanoid robots like the **Tesla Bot** to handle repetitive or dangerous work.

Overall, AI-driven robotics represents a major technological leap, turning long-imagined mechanical beings into practical, intelligent systems.

Computer vision

Computer vision is an AI field that enables computers to extract meaningful information from images and videos using machine learning and neural networks. Conceptually, if AI is the brain, computer vision acts as the eyes.

Computers process visual input from:

- **Images** — single static snapshots (simpler to analyze)
- **Videos** — sequences of frames over time (more complex due to motion and continuity)

To understand visual data, several major model families are used:

Convolutional Neural Networks (CNNs) are the foundation of most computer vision systems. They handle high-dimensional image data efficiently and learn spatial hierarchies—progressively detecting simple features (like edges) in early layers and complex objects in deeper layers.

Transformers can also be applied to vision tasks and are increasingly important, especially in modern generative AI systems.

Generative Adversarial Networks (GANs) are mainly used to generate realistic synthetic images.

Specialized architectures include:

- **U-Net** — widely used for medical image segmentation
- **EfficientNet** — focuses on computationally efficient model scaling

Computer vision has broad real-world applications, including:

- Self-driving vehicles
- Medical imaging
- Security and surveillance
- Robotics
- Face recognition software
 - It's just a model built into a software product without a robotic body.
- Virtual reality (education, entertainment, communication)

Overall, computer vision enables machines to interpret visual information from the real world and is a key driver of many modern AI advancements.

Traditional ML

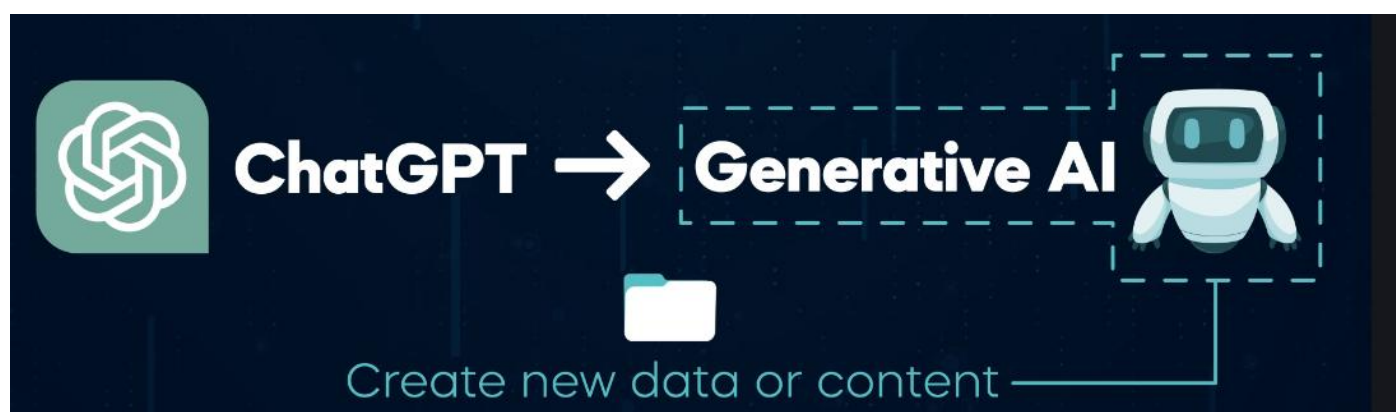
While high-profile AI products like ChatGPT and self-driving cars attract attention, most real business value from AI comes from practical, less glamorous applications.

Machine learning is widely used across industries:

- **Finance** — fraud detection and credit risk prediction
- **Insurance** — more accurate pricing of policies
- **Retail** — demand forecasting and inventory optimization
- **E-commerce** — dynamic pricing, conversion optimization, and product recommendations (e.g., Amazon)

These use cases have been delivering value for more than a decade. The key takeaway is that although general AI is exciting, traditional, domain-focused AI applications remain the primary drivers of measurable business impact.

Generative AI

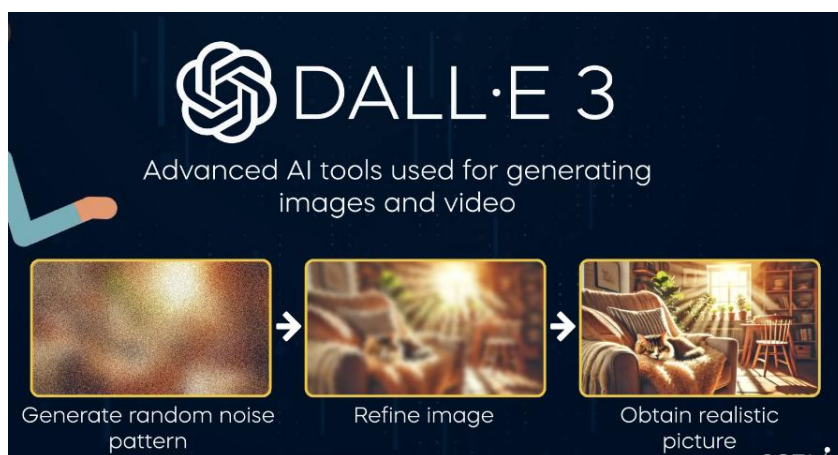


Generative AI is the branch of AI that creates new content rather than only analyzing existing data. It gained massive attention with tools like **ChatGPT**, which generates unique responses in real time based on patterns learned from large text datasets.

The defining capability of generative AI is producing novel outputs across formats such as text, images, video, audio, code, and 3D content. For example, image models like **DALL-E** start from random noise and progressively refine it into realistic images.

Several technical approaches power generative AI:

- **Large Language Models (LLMs)** — neural networks trained on massive text corpora to predict word sequences; foundation of modern chatbots.
- **Diffusion models** — generate images or video by gradually denoising random noise into structured visuals.

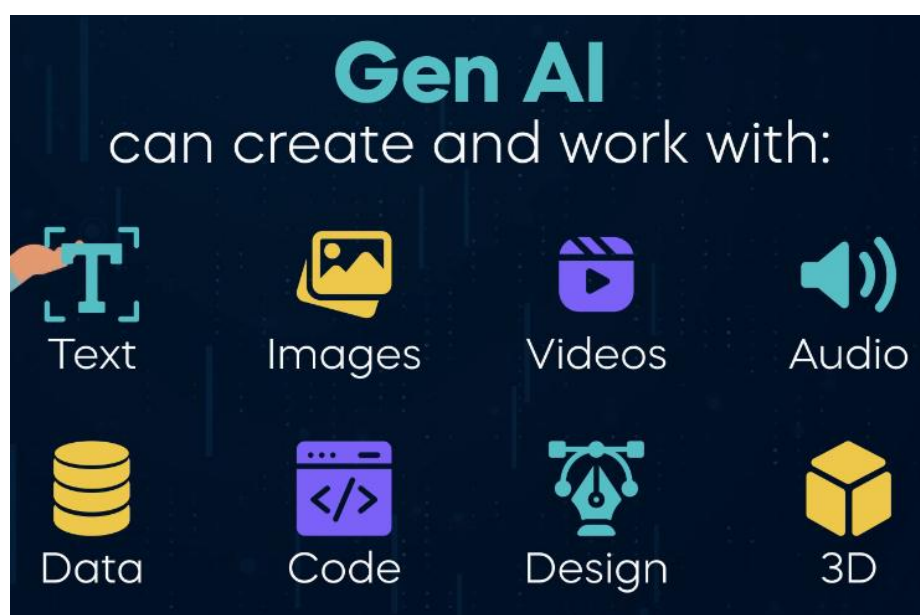


- **Generative Adversarial Networks (GANs)** — use a generator and discriminator competing to produce highly realistic content.



- **Neural Radiance Fields (NeRFs)** — specialized for 3D scene modeling.
- **Hybrid models** — combine multiple techniques (e.g., LLMs + GANs).

Generative AI is currently attracting heavy global investment and is expected to significantly transform businesses. Understanding how these systems work is becoming increasingly important as the technology rapidly advances.



Section 5: Intro to AI Module: Understanding Generative AI

The rise of Gen AI: Introducing ChatGPT

OpenAI released **ChatGPT 3.5** on November 30, 2022, as a dialogue-optimized text generation model. The launch was extraordinarily successful, gaining **over 1 million users in one week** and reaching **100 million users within two months**, making it the fastest-growing consumer application at the time (faster than TikTok).

The rapid adoption was driven by the model's practical usefulness. Users quickly discovered it could assist with everyday work such as:

- Summarizing text
- Answering technical questions
- Helping with professional and academic tasks

The later release of **GPT-4.0** demonstrated a significant performance improvement over GPT-3.5, solving many tasks more accurately and effectively. The trend suggests future versions will continue to improve.

The course segment aims to explore the **technical foundations behind ChatGPT and large language models**, emphasizing that generative AI is expected to remain an important technology.

Early approaches to Natural Language Processing

Natural Language Processing (NLP) is a branch of computer science that enables computers to understand, interpret, and generate human language. The field began in the **1950s** and has evolved through major phases.

1. Rule-based NLP (1950s)

Early NLP systems relied on manually written grammar rules. Developers explicitly told the computer how to interpret language.

- **Example:** A rule might say: sentences starting with “*Can you...*” or “*Will you...*” are questions.
 - Input: “*Can you help me?*”
 - Output: classified as a question.

These systems were simple but rigid and difficult to scale.

2. Statistical NLP (late 1980s–1990s)

The field shifted toward probabilistic, data-driven methods. Instead of fixed rules, models learned patterns from large annotated datasets.

- **Example problem:** Determine whether the word “**can**” is a noun or a verb.
 - “*I can swim*” → **verb** (ability)
 - “*Give me a soda can*” → **noun** (container)

Researchers collected many labeled sentences and analyzed surrounding words. Patterns such as:

- Words like “**I**” or “**you**” near *can* → likely a **verb**
- Words like “**soda**” near *can* → likely a **noun**

Using these frequencies, they built models that estimate the **probability** of the correct meaning in new sentences.

Key takeaway: NLP evolved from manual rule systems to statistical, data-driven approaches, which paved the way for modern machine learning and deep learning methods used in today's language AI.

Recent NLP advancements

In the **2000s**, NLP advanced significantly when statistical methods were combined with **machine learning**. A major breakthrough was the introduction of **vector embeddings**, which convert words and sentences into numerical vectors that capture meaning and relationships.

Vector embeddings allow complex data like text, images, and audio to be represented mathematically. In NLP, each word is mapped to a point in a **high-dimensional space** (often hundreds or thousands of dimensions). This high dimensionality is necessary because human language is too complex to represent in just 2–3 dimensions.

- **Example:**
Words with similar meanings end up close together in vector space.
 - “king” and “queen” → close
 - “king” and “banana” → far apart

This enables **semantic similarity search** — retrieving text based on meaning rather than exact keywords.

Impact of machine learning models:

New ML models began analyzing large text datasets using embeddings. These models could detect subtle language cues such as **sentiment, sarcasm, and irony**, which earlier statistical methods struggled to capture.

- **Example:**
Sentence: *“Great, my phone died again.”*
Basic methods → may detect positive word “Great”
ML with embeddings → understands negative sentiment

Neural network revolution (2010s):

Deep neural networks with many layers dramatically improved NLP tasks because they can learn complex patterns.

This led to major improvements in:

- Machine translation
- Speech recognition
- Text generation

Transformer architecture (2018):

Transformers became a landmark innovation in NLP. Their design enabled efficient handling of long text context and powered modern **large language models (LLMs)** such as GPT and Gemini.

Key takeaway

NLP evolved from simple statistics to powerful deep learning systems thanks to vector embeddings, neural networks, and transformers—foundations that made today’s large language models possible.

From Language Models to Large Language Models (LLMs)

The speaker uses a **team word-association game** to explain how language models work.

Game analogy:

Players give a single related word to help their teammate guess a secret word. Success depends on choosing the word that maximizes the chance of the correct guess.

- **Example:**

Secret word: *banana*

- Saying “**yellow**” → weak clue (many yellow things)
- Saying “**peel**” → stronger clue (more specific association)

Sometimes multiple hints improve accuracy:

- **Example:**

Secret word: *grapefruit*

- First hint: “**citrus**”
- Second hint: “**bitter**” or “**pink**”

Key idea: Context and probability determine the best word choice.

How this relates to language models

Language models work similarly — they **predict the most probable next word** based on context.

- **Example:**

Input: “*To be or not to be, that is the ____*”

Model predicts: “**question**”

Language models are therefore **probabilistic systems** that fill in missing words using learned patterns from large text datasets.

Two types of language models

1. Masked language models (MLM)

- Can predict a missing word anywhere in a sentence
- Use both left and right context
- **Example:**
“*Water ____ at zero degrees Celsius.*”
→ Model predicts “**freezes**”

2. Autoregressive language models

- Predict the **next** word only
- Use only previous context
- GPT models belong here
- **Example:**
“Water freezes at zero degrees ____”
 → Model predicts **“Celsius”**

How models learn

Language models analyze **massive text datasets** to learn word relationships and patterns. Over time they build a **probabilistic map of language**, enabling coherent and context-aware text generation.

Because they generate new text rather than retrieve fixed answers, they are part of **generative AI**.

About Large Language Models (LLMs)

- No strict definition of “large”
- Generally refers to models trained on **huge datasets and many parameters**
- Scale keeps increasing

Examples:

- GPT-3 → ~175 billion parameters
- GPT-4 → over 1 trillion parameters
- Future models expected to be even larger

Key takeaway:

Language models work by predicting the most likely next word using context

The efficiency of LLM training, Supervised vs Semi-supervised Learning

1. Supervised Machine Learning

Definition

Supervised learning is a machine learning approach where the model is trained using **labeled data**. Each input in the dataset has a corresponding **correct output label**.

The model learns a mapping:

Input → Correct Output

Example: Sentiment Analysis

Suppose we want a model that classifies customer reviews as:

- Positive
- Negative

We create a dataset like this:

Review	Label
"This product is amazing!"	Positive
"Worst purchase ever."	Negative
"Good quality and fast delivery."	Positive
"Completely disappointed."	Negative

If we collect **100,000 reviews**, humans must read and label them manually.

How the Model Learns

The model analyzes patterns such as:

Positive words:

- amazing
- great
- love
- excellent

Negative words:

- terrible
- bad
- worst
- disappointed

Using these patterns, it learns to classify **new unseen reviews**.

2. The Major Problem with Supervised Learning

Labeling Data is Expensive

Human labeling requires time and money.

Example cost calculation:

Cost per review = **\$0.30**

Dataset size = **100,000 reviews**

Total cost:

$$100,000 \times 0.30 = \$30,000$$

Cost Increases with Complexity

Simple data (short reviews) may cost **\$0.30 per label**.

But complex data requires expert knowledge:

Data Type	Estimated Label Cost
Customer reviews	\$0.30
News articles	\$5–\$10
Legal documents	\$20+
Medical records	\$30+

If labeling medical records:

$$100,000 \times \$30 = \$3,000,000$$

So large datasets become **extremely expensive**.

Scalability Problem

Supervised learning does **not scale well** when:

- Data size is huge
- Labels require experts
- Data complexity increases

3. Why Supervised Learning Cannot Train Large Language Models Alone

People often say:

ChatGPT has “read the entire internet”.

Imagine labeling the entire internet manually.

Problems:

1. **Impossible labor requirements**
2. **Huge cost**
3. **Human bias in labeling**
4. **Slow dataset creation**

Therefore, supervised learning alone **cannot train large language models (LLMs)**.

4. Unsupervised Learning

Definition

Unsupervised learning uses **unlabeled data**.

The algorithm tries to discover **hidden patterns or structures** in the data.

Example tasks:

- Clustering similar documents
- Finding hidden topics
- Detecting anomalies

Example

Given these sentences:

- "I love this movie"
- "This phone is amazing"
- "Worst service ever"
- "I hate this product"

An unsupervised model might group them into clusters like:

Cluster 1:

- positive sentiment

Cluster 2:

- negative sentiment

But **no labels are provided**.

Limitations of Unsupervised Learning for Language

Language is highly contextual.

Example sentence:

The bank was crowded.

"Bank" could mean:

- financial institution
- river bank

The meaning depends on **context**.

Language also builds context across:

- words
- sentences
- paragraphs

Unsupervised learning alone struggles because:

1. It lacks a **clear training objective**
2. It doesn't necessarily learn **language prediction**
3. It may miss **important context relationships**

5. The Breakthrough: Self-Supervised Learning

To solve these problems, researchers developed **self-supervised learning**.

Definition

Self-supervised learning is a method where:

- **The data generates its own labels automatically**
- The model learns from **unlabeled data**

It is essentially a **hybrid between supervised and unsupervised learning**.

How Self-Supervised Learning Works

Instead of humans labeling data, the model creates labels from the text itself.

Example sentence:

The cat sat on the mat.

Training task:

Input:

The cat sat on the ____

Label:

mat

The model learns to **predict the next word**.

Another example:

Input:

Machine learning is transforming ____

Expected prediction:

technology

The label is automatically derived from the sentence.

Why This Approach Scales

Advantages:

1. No Human Labeling Required

The internet itself becomes the dataset.

2. Massive Training Data

Models can train on:

- books
- articles
- research papers
- websites
- code repositories

3. Lower Cost

Instead of paying humans to label data, the model learns **directly from raw text**.

6. Why Self-Supervised Learning Works Well for Language

Language models learn:

Context

Words are interpreted based on surrounding words.

Example:

He deposited money in the bank.
They sat on the river bank.

The model learns contextual meaning automatically.

Grammar

The model sees billions of sentences and learns patterns like:

- subject–verb agreement
- sentence structure
- punctuation usage

Semantics

It learns relationships between concepts:

```
doctor → hospital
king → queen
Paris → France
```

7. Self-Supervised Learning in Large Language Models

This approach enabled the creation of modern LLMs such as:

- ChatGPT
- GPT-4
- BERT

These models are trained on **massive unlabeled text datasets** using tasks such as:

- next-word prediction
- masked word prediction
- sentence completion

8. Summary Comparison

Approach	Data Type	Label Source	Scalability
Supervised Learning	Labeled	Humans	Low
Unsupervised Learning	Unlabeled	None	Medium
Self-Supervised Learning	Unlabeled	Data itself	Very High

9. Key Takeaways

1. **Supervised learning** requires labeled data but becomes extremely expensive at scale.
2. **Unsupervised learning** works without labels but lacks clear training objectives for language tasks.
3. **Self-supervised learning** solves both problems by allowing models to generate labels automatically from raw data.

4. This approach enabled the development of **modern large language models** capable of understanding and generating human language.
5. The next major breakthrough that made these systems practical was the **Transformer architecture**, which allows models to process massive text datasets efficiently.

From N-Grams to RNNs to Transformers: The Evolution of NLP

Evolution of Language Models: From N-grams to Transformers

Language models are systems designed to **predict the next word in a sequence** or estimate the probability of a sentence. Over time, language modeling has evolved significantly—from simple statistical methods to powerful neural architectures used in modern **Large Language Models (LLMs)** such as ChatGPT.

This section explains **four major techniques used in language modeling**, how they work, and how each solved the limitations of the previous approach.

1. N-gram Models

Concept

An **N-gram model** predicts the probability of a word based on the **previous N-1 words**.

General idea:

$$P(w_n \mid w_{n-1}, w_{n-2}, \dots, w_{n-(n-1)})$$

Where:

- **N** = number of words considered
- **N-1 words** = context used for prediction

Types of N-grams

1. Unigram (N = 1)

A **unigram model** predicts words **without using any context**.

Example sentence:

"My favorite sport is ____"

The model ignores the sentence context and simply predicts the **most frequent word in the entire training dataset**.

Possible predictions:

- "the"
- "and"
- "is"

This approach is clearly ineffective because it **does not consider sentence meaning**.

2. Bigram (N = 2)

A **bigram model** uses **one previous word** to predict the next.

Example:

"My favorite sport is ____"

The model looks at the previous word:

is

It searches the training data for the most common word following **"is"**.

Possible predictions:

- "a"
- "the"
- "basketball"

3. Trigram (N = 3)

A **trigram model** uses **two previous words**.

Example:

"My favorite sport is ____"

Context used:

sport is

The model checks the dataset for the most common word that follows **"sport is"**.

Possible prediction:

- "basketball"

Limitations of N-gram Models

Despite being foundational, N-grams have serious weaknesses:

1. **Limited context**
 - Only considers a small number of previous words.
2. **Data sparsity**
 - Many word combinations never appear in training data.
3. **No understanding of meaning**
 - Predictions are purely based on **frequency counts**.

Because of these issues, researchers turned to **neural networks**.

2. Recurrent Neural Networks (RNNs)

Concept

Recurrent Neural Networks (RNNs) were designed to process **sequential data**, making them suitable for natural language.

Unlike N-grams, RNNs:

- Process words **one at a time**
- Maintain a **hidden state (memory)** that carries information from previous words.

Example processing sequence:

My → favorite → sport → is → basketball

At each step, the network:

1. Reads the current word
2. Updates its internal memory
3. Uses that memory to predict the next word

Key Advantage

RNNs capture **word order and context**, which is essential for language understanding.

For example:

I went to the bank to deposit money.

Here, the word **bank** refers to a financial institution, and the surrounding words help determine that.

Major Problem: Vanishing Gradient

RNNs struggle with **long sequences**.

As sentences get longer:

- Early information gradually **fades away**
- The model forgets earlier words

This issue is known as the **Vanishing Gradient Problem**.

Example:

The book that I borrowed from the library yesterday was very interesting.

An RNN might forget **"book"** by the time it reaches the end.

To address this limitation, researchers developed **LSTM networks**.

3. Long Short-Term Memory Networks (LSTMs)

Concept

LSTM networks are an improved version of RNNs designed to handle **long-term dependencies**.

They introduce a **gate architecture** that controls information flow.

LSTM Gates

Three gates regulate the memory cell:

1. Forget Gate

Decides **what information should be discarded**.

2. Input Gate

Determines **what new information should be stored**.

3. Output Gate

Controls **what information should influence the next step**.

Advantage

LSTMs can remember important information over long distances in text.

Example:

The movie that we watched last night was incredibly entertaining.

The model remembers "**movie**" while processing later words.

Limitations

Although LSTMs are powerful, they have several drawbacks:

1. **High computational cost**
2. **Slow training**
3. **Sequential processing** (words must be processed one after another)

These issues made them difficult to scale for very large datasets.

The breakthrough came in **2017**.

4. Transformers and the Attention Mechanism

In 2017, researchers published the landmark paper:

Attention Is All You Need

This paper introduced the **Transformer architecture**, which revolutionized natural language processing.

Core Idea: Attention

The **attention mechanism** allows the model to determine **which words in a sentence are most important** when making predictions.

Instead of processing text sequentially like RNNs, transformers:

- Examine the **entire sentence at once**
- Assign **attention scores** to each word

Words with higher scores receive more focus.

Example

Sentence:

The animal didn't cross the street because it was too tired.

The word "it" refers to **animal**.

The attention mechanism helps the model link these words, even if they are far apart.

Advantages of Transformers

Transformers solved several limitations of earlier models:

1. Parallel Processing

Unlike RNNs and LSTMs, transformers can process **all words simultaneously**, making training much faster.

2. Long-Range Context

Attention allows the model to capture **relationships between distant words**.

3. Scalability

Transformers scale efficiently to **massive datasets and models**.

This capability enabled the development of modern **Large Language Models (LLMs)** such as:

- ChatGPT
- GPT-4
- BERT

Summary: Evolution of Language Models

STAGE	TECHNIQUE	KEY IDEA	LIMITATION
1	N-grams	Predict next word using previous few words	Very limited context
2	RNNs	Use sequential memory to capture context	Forget long sequences
3	LSTMs	Gate architecture for long-term memory	Slow and expensive
4	Transformers	Attention mechanism to focus on relevant words	High compute requirements

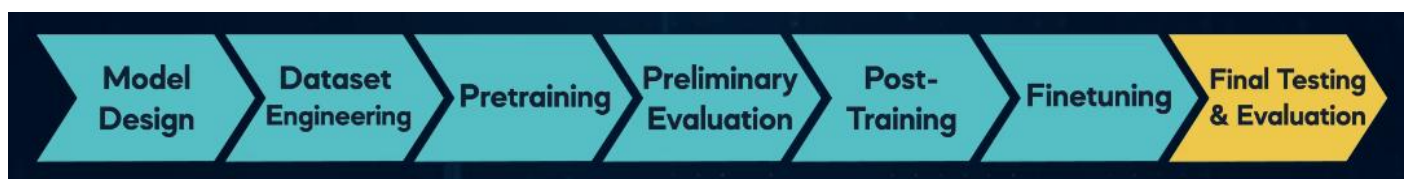
Key Takeaway

Language modeling evolved through **four major breakthroughs**:

1. **Statistical methods (N-grams)**
2. **Sequential neural models (RNNs)**
3. **Memory-enhanced networks (LSTMs)**
4. **Attention-based architectures (Transformers)**

The **transformer architecture** ultimately made modern **Large Language Models** possible.

Phases in building LLMs



Stages of Building a Large Language Model (LLM)

Modern AI systems such as ChatGPT are created through a multi-stage development process involving model architecture decisions, massive datasets, training procedures, and rigorous evaluation.

Although these stages are usually presented sequentially for explanation, in real-world development they often **overlap and iterate**.

The main stages include:

- Model Design
- Dataset Engineering
- Pre-Training
- Preliminary Evaluation
- Post-Training
- Fine-Tuning
- Final Testing and Evaluation

Model Design

The model design phase establishes the technical foundation of the language model.

AI researchers and engineers decide:

- the neural network architecture
- the number of layers (model depth)
- the number of parameters
- the training objectives

Common neural network architectures used in natural language processing include:

- Transformers
- Recurrent Neural Networks (RNNs)
- Convolutional Neural Networks (CNNs)

Most modern large language models are based on the Transformer architecture introduced in the paper **Attention Is All You Need**.

Important design choices include:

- **Architecture**
Determines how the model processes language and relationships between words.
- **Depth** (number of layers)
More layers allow the network to capture more complex patterns.
- **Parameter count**
Large models can contain **billions or even trillions of parameters**, which increases capability but also increases computational cost.

These design decisions define:

- what the model can do
- how expensive it is to train
- how fast it runs

Dataset Engineering

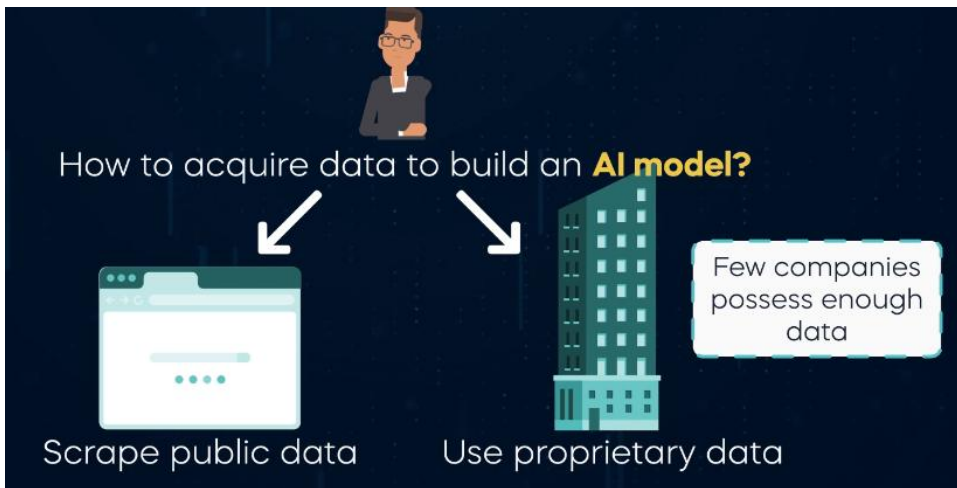
A common principle in machine learning states:

“A model is only as good as the data used to train it.”

Dataset engineering focuses on preparing training data.

Major tasks include:

- collecting data
- cleaning and filtering data
- removing duplicates and noise
- structuring the dataset for training



Two primary methods are used to acquire training data.

Public Data

This data is scraped from publicly available sources such as:

- websites
- books
- research papers
- Wikipedia
- online forums

Proprietary Data

This is privately owned data such as:

- company documents
- internal knowledge bases
- customer service conversations
- domain-specific industry datasets

Organizations that own large proprietary datasets often gain a **competitive advantage** when building AI systems.

Ethical Considerations in Dataset Engineering

Engineers must carefully address ethical challenges when constructing datasets.

Bias

Training data may include harmful biases related to:

- gender
- culture
- politics
- race

If not handled properly, these biases may appear in model outputs.

Data Diversity

Datasets should represent multiple perspectives, languages, and cultures to reduce biased behavior.

Responsible dataset design helps produce safer AI systems.

Pre-Training

After the dataset is prepared, the model enters the pre-training stage.

Pre-training involves training the neural network on a **massive corpus of text**.

Typical learning objectives include:

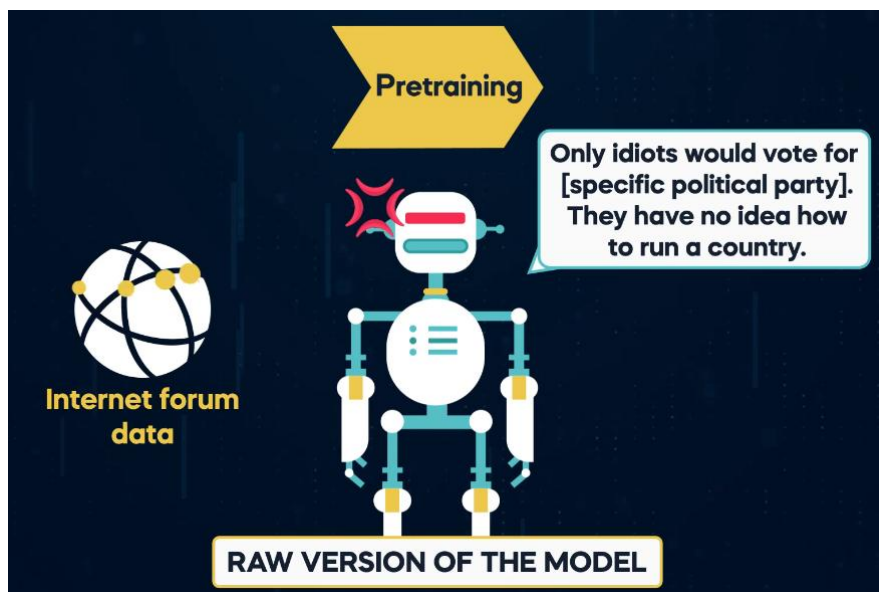
- predicting the next word in a sentence
- predicting masked or hidden words

During this process the model learns:

- grammar
- sentence structure
- relationships between words
- patterns in human language

The result is a **base model**.

However, this raw model may still generate problematic responses depending on the training data.



Examples of issues include:

- toxic language
- biased responses
- misinformation

This occurs because the model reflects patterns present in the training data.

Preliminary Evaluation

Once pre-training is complete, developers perform an initial evaluation to assess the model's behavior.

This stage helps identify:

- knowledge limitations
- hallucinations
- bias issues
- incorrect or harmful responses

For example, if a model is trained heavily on internet forums, it may adopt:

- informal tone
- offensive language
- unreliable information

These problems must be identified before further improvements are made.

Post-Training

Post-training improves the behavior of the base model.

This phase typically includes two major steps.

Supervised Fine-Tuning (SFT)

In this step, human experts create high-quality example responses.

The model is trained to imitate these responses.

Example:

Customer:

“My internet connection is not working.”

Desired chatbot response:

“I’m sorry you're experiencing connection issues. Let me help you troubleshoot the problem.”

This process teaches the model how to produce:

- polite responses
- structured answers
- helpful information

Human Feedback

Human reviewers evaluate model responses and provide corrections or rankings.

This feedback helps guide the model toward:

- safer outputs
- more accurate answers
- more helpful responses

Fine-Tuning

Fine-tuning modifies the model's internal weights to improve performance in specific tasks.

Common objectives include:

Domain Adaptation

Adapting a general language model for specialized industries such as:

- healthcare
- finance
- law
- customer service

Performance Optimization

Fine-tuning can also make a model:

- smaller
- faster
- cheaper to deploy

However, specialization may sometimes reduce the model's general knowledge capabilities.

Final Testing and Evaluation

The final phase involves strict testing before the model is deployed to real users.

Developers evaluate the system from the perspective of an end user.

Important evaluation criteria include:

Response Quality

Are answers clear, helpful, and well structured?

Accuracy

Does the model produce factually correct information?

Speed

How quickly can the model generate responses?

Safety and Ethics

Does the model avoid harmful or biased outputs?

Only after passing these evaluations can the model be safely released for public use.

Summary of the LLM Development Pipeline

Model Design

Defines the architecture and model capacity.

Dataset Engineering

Collects and prepares large amounts of training data.

Pre-Training

Trains the model on large text datasets to learn language patterns.

Preliminary Evaluation

Identifies weaknesses and potential risks.

Post-Training

Improves behavior using supervised data and human feedback.

Fine-Tuning

Adapts the model to specialized tasks or improves efficiency.

Final Testing and Evaluation

Ensures the model is accurate, safe, and reliable before deployment

Prompt engineering vs Fine-tuning vs RAG: Techniques for AI optimization

PROMPT ENGINEERING	RAG Retrieval-augmented generation	FINE-TUNING
Explain to the model how to behave	Expand context by attaching a library	Additional training for an already trained model
✗ Weight change ✓ Verbal instructions	✗ Weight change ✓ Attach a database	✓ Weight change ✓ Extra data
EASY ↓ Does not change; It adapts	MEDIUM ↓ Model does not change	HARD ↓ Model changes; Can be expensive

Introduction

When developing applications with Large Language Models (LLMs), developers often try to improve the quality, relevance, and reliability of the model's responses. Three important techniques used for this purpose are:

- **Prompt Engineering**
- **Retrieval-Augmented Generation (RAG)**
- **Fine-Tuning**

These terms are often used interchangeably, but they actually represent **three different ways of improving an AI system**. Each method works at a different level and involves different levels of complexity, cost, and infrastructure.

Understanding these techniques is essential for designing effective AI-powered applications.

Prompt Engineering

Prompt engineering is the process of **guiding an LLM using carefully designed prompts**. A prompt is simply the input text given to the model.

In this technique:

- The **model itself is not modified**
- **No additional training is performed**
- The **weights of the model remain unchanged**

Instead, developers influence the model's output by giving **clear instructions, context, examples, or constraints** within the prompt.

How Prompt Engineering Works

The developer structures the prompt to help the model understand:

- What task it should perform
- What format the answer should follow
- What tone or style to use
- What context should influence the response

For example, instead of writing a vague prompt like:

Explain recursion.

A better prompt-engineered version would be:

You are a computer science tutor. Explain recursion to a beginner using a simple analogy and a small C++ example.

The model remains the same, but the **quality of the response improves** because the instructions are clearer.

Advantages of Prompt Engineering

- Easy to implement
- Requires no additional infrastructure
- Very low cost
- Allows quick experimentation and iteration

Limitations of Prompt Engineering

- Cannot expand the model's knowledge
- Improvements are limited to better instructions
- Complex behavior may require many prompt iterations

Prompt engineering is typically **the first technique developers use** when improving an AI system.

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) improves an LLM by **connecting it to an external knowledge source**, such as a database or document repository.

In RAG systems:

- The **model weights remain unchanged**
- External information is retrieved from a database
- That information is included in the prompt as additional context

This allows the model to use **information that was not part of its original training data**.

How RAG Works

A typical RAG workflow consists of three steps:

1. **User Query**
The user asks a question.
2. **Retrieval Step**
The system searches a database or knowledge base for relevant information.
3. **Augmented Prompt**
The retrieved information is inserted into the prompt before sending it to the LLM.

The model then generates an answer using both:

- Its pretrained knowledge
- The retrieved context

Example

Imagine building an AI assistant for company documentation.

A user asks:

How do I deploy our internal API?

The system retrieves the relevant documentation and constructs a prompt such as:

Use the following documentation to answer the question:
[retrieved documentation]

Question: How do I deploy our internal API?

The model then produces a response based on the retrieved information.

Advantages of RAG

- Allows models to use **up-to-date information**
- Enables integration with **private or proprietary data**
- Reduces hallucinations
- Does not require retraining the model

Limitations of RAG

- Requires additional infrastructure (retrieval system, database)
- Quality depends on the relevance of retrieved documents
- System architecture becomes more complex

RAG is widely used in modern AI applications such as enterprise knowledge assistants and document-based question-answering systems.

Fine-Tuning

Fine-tuning is the process of **retraining a pretrained model using additional training data**, which updates the model's internal parameters (weights).

Unlike prompt engineering and RAG, **fine-tuning actually changes the model itself**.

How Fine-Tuning Works

The process typically involves:

1. Starting with a pretrained LLM (for example from OpenAI).
2. Preparing a dataset that demonstrates the desired behavior.
3. Training the model further on this dataset.
4. Updating the model's weights based on the new data.

The result is a model that is **better specialized for a particular task or domain**.

Example

Suppose you want an AI system that specializes in:

- Legal document analysis

- Medical report interpretation
- Customer support responses

You could fine-tune the model using thousands of examples from that domain so that it learns the specific terminology, reasoning patterns, and response style required.

Advantages of Fine-Tuning

- Improves performance for specialized tasks
- Allows domain-specific knowledge and behavior
- Can make responses more consistent and reliable
- Reduces the need for complex prompts

Limitations of Fine-Tuning

- Requires labeled training data
- Computationally expensive
- Slower iteration cycles
- Requires machine learning infrastructure

Fine-tuning is typically used **only when prompt engineering and RAG are insufficient**.

Comparison of the Three Techniques

Prompt Engineering

- **What changes:** Only the prompt instructions
- **Model weights:** Unchanged
- **Data required:** None
- **Cost:** Very low
- **Iteration speed:** Very fast

Retrieval-Augmented Generation (RAG)

- **What changes:** External knowledge source added
- **Model weights:** Unchanged
- **Data required:** External documents or databases
- **Cost:** Moderate infrastructure cost
- **Iteration speed:** Medium

Fine-Tuning

- **What changes:** Model parameters (weights)
- **Model weights:** Updated through training
- **Data required:** Labeled training datasets
- **Cost:** High computational cost
- **Iteration speed:** Slow

Practical Example: Interview Simulator

Consider a team building an AI-powered interview simulator using an LLM.

Prompt Engineering in the Project

Developers wrote prompts instructing the model to:

- Ask coding questions
- Follow different interview stages
- Maintain a professional tone
- Evaluate the candidate's answers
- Provide feedback

This required multiple iterations to achieve the desired behavior.

Using RAG

The team also connected the system to a **database of interview questions**.

When a user selected an interview type (for example, algorithms or system design), the system retrieved relevant questions from the database and inserted them into the prompt.

This allowed the model to generate **context-aware interview questions**.

Why Fine-Tuning Was Not Used

Fine-tuning would have required retraining the model to:

- Respond faster
- Specialize in interview-style conversations

However, this was unnecessary because:

- Prompt engineering already controlled the interview behavior.
- RAG supplied the required interview questions.
- The model needed flexibility to handle different interview scenarios.

Key Takeaways

- **Prompt Engineering** improves outputs by giving better instructions to the model.
- **RAG** improves outputs by providing additional knowledge from external sources.
- **Fine-Tuning** improves outputs by retraining the model with new data.

In most real-world AI systems, developers typically follow this progression:

1. Start with **prompt engineering**.
2. Add **RAG** when external knowledge is required.
3. Use **fine-tuning** only when deeper specialization is necessary.

Understanding these techniques is a fundamental skill in modern AI application development.

The importance of foundation models

Introduction

For many years, machine learning systems were designed to perform **very specific and narrow tasks**. Each model was built for a particular problem and could not easily be reused for other types of tasks. However, the development of **Large Language Models (LLMs)** has changed this paradigm significantly.

LLMs introduced a shift from **narrow AI systems** to **general-purpose AI systems** capable of performing a wide variety of tasks across multiple domains.

Traditional Machine Learning Models: Narrow AI

Earlier machine learning systems were typically designed to specialize in **one specific application**. These systems performed extremely well within their designated domain but could not easily adapt to other types of problems.

Examples include:

Image Recognition Models

These models were trained specifically to **identify objects in images**. Typical applications included:

- Detecting faces in photos
- Identifying objects such as cars, animals, or buildings
- Medical image analysis

The model's knowledge was limited strictly to **visual data processing**.

Speech Recognition Systems

Speech recognition models focused on **converting spoken language into written text**.

Common uses included:

- Voice assistants
- Transcription systems
- Voice command interfaces

These systems specialized in **audio processing and language transcription**.

Natural Language Processing (NLP) Models

Traditional NLP models were designed for **specific language tasks**, such as:

- Sentiment analysis
- Language translation
- Named entity recognition
- Spam detection

Each model typically handled **only one linguistic task**.

Time Series Models

Time series models focused on **predicting patterns over time**.

Typical applications included:

- Stock price prediction
- Market volatility forecasting
- Weather prediction
- Demand forecasting

Again, these systems were designed for **a single category of problems**.

Key Limitation

The major limitation of these models was **task specialization**.

A model trained for one task generally could **not perform another task without retraining from scratch**.

The Shift Introduced by Large Language Models (LLMs)

The emergence of LLMs created a major transition in artificial intelligence.

Large Language Models have:

- **Complex neural network architectures**
- **Massive training datasets**
- **Billions or trillions of parameters**

Because of this scale, they are capable of performing **general-purpose tasks** instead of narrow ones.

Unlike earlier models, LLMs can perform tasks such as:

- Writing text
- Answering questions
- Summarizing documents
- Generating code
- Translating languages
- Explaining technical concepts

With techniques such as **prompt engineering** and **fine-tuning**, these models can adapt to many different domains.

From Text Models to Multimodal Systems

Initially, LLMs were designed as **text-to-text systems**.

This means:

- The input was text.
- The output was also text.

For example:

Input:

“Explain recursion in C++.”

Output:

A textual explanation of recursion.

However, the capabilities of these models expanded rapidly.

Modern AI systems can now process and generate multiple types of data, including:

- Code
- Images
- Videos
- Audio
- Structured files

For example, modern systems can:

- Generate programming code
- Read and analyze **PDF documents**
- Interpret **Excel spreadsheets**
- Understand **images**
- Generate **visual content**

This evolution marked a transition from **pure language models** to **multimodal AI systems**.

Multimodal AI

A **multimodal system** can understand and generate multiple forms of data simultaneously.

Examples of modalities include:

- Text
- Images
- Audio
- Video
- Structured data

For instance, a multimodal AI system could:

- Analyze an image and describe it in text
- Read a document and answer questions about it
- Generate images based on text descriptions
- Convert spoken speech into text and summarize it

These capabilities significantly expand the range of applications AI systems can support.

The Concept of Foundation Models

As AI systems became larger and more versatile, researchers began referring to them as **foundation models**.

The term reflects the idea that these models serve as a **base layer upon which many applications can be built**.

A foundation model is characterized by:

- Extremely large scale
- General-purpose capabilities
- Ability to adapt to multiple tasks

Developers do not usually build applications by training such models from scratch. Instead, they **build applications on top of these foundational systems**.

Major organizations developing foundation models include:

- OpenAI
- Google
- Meta
- Microsoft

Why Only a Few Organizations Can Build Foundation Models

Building a foundation model requires enormous resources, including:

- Massive computational infrastructure
- Huge training datasets
- Large research teams
- Significant financial investment

According to Sam Altman, CEO of OpenAI, only a **small number of organizations** may be able to build such models in the future.

These organizations include:

- Large technology companies
- Major AI research labs
- Some government-backed initiatives

Because of the cost and complexity involved, most companies will **not build their own foundation models**.

How Most Companies Will Use Foundation Models

Instead of training their own foundation models, most organizations will:

1. Use existing foundation models.
2. Build specialized applications on top of them.
3. Customize them using techniques such as:
 - Prompt engineering
 - Retrieval-Augmented Generation (RAG)
 - Fine-tuning

This approach allows companies to **leverage powerful AI systems without needing enormous infrastructure**.

The Role of Open Source

While large technology companies currently dominate the development of foundation models, the **open-source AI community** is also making significant progress.

Open-source models could eventually provide alternatives that are:

- More accessible
- More customizable
- Less dependent on large technology corporations

The long-term balance between **big tech models and open-source models** remains an important topic in the AI ecosystem.

Key Takeaways

- Traditional machine learning systems were **narrow and task-specific**.
- Large Language Models introduced **general-purpose AI capabilities**.
- Modern AI systems evolved from **text-only models to multimodal systems**.
- These large general-purpose systems are called **foundation models**.
- Only a small number of organizations currently have the resources to build them.
- Most companies will instead **build applications on top of existing foundation models**.

Foundation models are becoming the **core infrastructure layer of modern AI systems**, enabling innovation across many industries and disciplines.

Buy vs Make: foundation models vs private models

Introduction

One of the classic strategic decisions in business management is the “**Buy vs. Make**” dilemma. Organizations must decide whether to **build a capability internally** or **acquire it from an external provider**. This decision becomes particularly important when dealing with advanced technologies such as Artificial Intelligence and Large Language Models (LLMs).

With the emergence of powerful foundation models, this traditional business decision has taken on a **new and unique dimension**.

Business Strategy Concept: Buy vs. Make Decision

In business strategy, the **buy vs. make decision** determines whether a company should:

Make (Build Internally)

Develop a capability using internal resources.

Buy (Outsource or Acquire)

Purchase the capability or service from external providers.

Guiding Principle in Business Strategy

Business schools and MBA programs often teach the following rule:

Companies should **retain internally all strategic, value-adding activities**, while outsourcing **non-core operational activities**.

Internal Strategic Activities (Make)

Companies typically keep the following capabilities internally:

- Core technologies
- Key intellectual property
- Strategic product capabilities
- Competitive differentiators

These activities directly contribute to a company’s **long-term competitive advantage**.

External Operational Activities (Buy)

Organizations frequently outsource tasks such as:

- IT infrastructure

- Payroll services
- Manufacturing components
- Customer support operations

These tasks are generally **efficiency-driven** rather than strategically critical.

Why Artificial Intelligence Changes the Traditional Model

When applying the buy vs. make framework to **AI and Large Language Models**, the situation becomes unusual.

Foundation models are:

- Extremely expensive to build
- Computationally intensive
- Technically complex
- Resource heavy

Only a small number of organizations currently have the **infrastructure, data, and expertise** required to train these models.

Major organizations developing foundation models include:

- OpenAI
- Google
- Meta
- Microsoft

Because of the enormous cost and complexity, **most companies cannot realistically build their own foundation models.**

Strategic Dilemma for Businesses

Many organizations recognize that **AI is becoming a core value-generating capability.** AI can influence:

- Product features
- Customer experience
- Automation
- Decision-making
- Competitive positioning

However, despite AI being strategically important, most companies **cannot build the underlying models themselves.**

The Contradiction

This creates a strategic contradiction:

- AI is **strategically critical** for business success.
- But the underlying technology **comes from external providers**.

This challenges the traditional rule that **strategic capabilities must always be built internally**.

Model-as-a-Service (MaaS)

To address this problem, AI providers offer **Model-as-a-Service (MaaS)** platforms.

Through APIs, companies can access powerful foundation models without building them from scratch.

For example, developers can integrate models such as GPT through APIs provided by OpenAI.

Typical API Interaction

Companies interact with the model in a simple workflow:

1. Send a request to the API
2. Provide a prompt or input data
3. Receive the AI-generated response

This model significantly **reduces the barrier to adopting AI technology**.

The Competitive Question

If every company can access the same AI models through APIs, an important question arises:

How will companies compete if they all use the same AI technology?

For example:

- Two companies use the same GPT API.
- Both access the same underlying model.
- Both generate text, code, or analysis.

In this situation, the **foundation model itself is no longer the differentiating factor**.

Instead, the competitive advantage shifts to **how the technology is applied**.

The Real Competitive Advantage: AI Adaptation

The key differentiator becomes the ability to **adapt foundation models to specific business use cases**.

This adaptation typically involves three main techniques.

Technique 1: Prompt Engineering

Prompt engineering involves **designing prompts that guide the model toward better responses**.

Common prompt engineering practices include:

- Structuring prompts clearly
- Providing contextual information
- Including examples
- Defining output formats

Skilled prompt engineering can significantly **improve AI output quality**.

Technique 2: Retrieval-Augmented Generation (RAG)

RAG enhances models by connecting them to **external knowledge sources**.

Typical sources include:

- Company databases
- Internal documents
- Product catalogs
- Knowledge bases

This allows the AI system to generate **company-specific responses instead of generic answers**.

Technique 3: Fine-Tuning

Fine-tuning involves **training the model further on domain-specific data**.

This allows organizations to specialize models for tasks such as:

- Customer support automation
- Legal document analysis
- Financial forecasting
- Software development assistance

Fine-tuning enables models to **adapt more deeply to specific domains**.

Rising Demand for AI Engineers

Because AI adaptation is becoming the main competitive factor, companies increasingly need professionals who can:

- Design effective prompts
- Build RAG systems
- Fine-tune models
- Integrate AI into products

As a result, demand for **AI engineers and applied AI specialists** is expected to grow significantly.

These engineers will focus less on training massive models and more on **engineering intelligent AI-powered applications**.

A New Paradigm in Technology Strategy

The rise of foundation models represents a **fundamental shift in how technological capabilities are developed**.

Traditional Technology Model

Companies build their own core technologies.

New AI Ecosystem Model

- A small number of organizations build foundation models.
- Most companies build applications **on top of these models**.

This layered structure resembles other technology ecosystems such as:

- Cloud computing platforms
- Operating systems
- Mobile app ecosystems

Foundation models are increasingly becoming the **infrastructure layer of modern AI applications**.

Key Takeaways

- The **buy vs. make decision** is a fundamental concept in business strategy.
- AI and foundation models introduce a unique challenge to this framework.
- Most companies cannot build their own large-scale AI models.
- Many businesses rely on **Model-as-a-Service platforms** from providers such as OpenAI.
- Competitive advantage will come from **how companies apply AI**, not from the base model itself.
- Skills such as **prompt engineering, RAG, and fine-tuning** will become critical for building successful AI products.